

Connecting timeout-service to Sentinel Ops

This walks through registering the deployed sample app as an application in Sentinel, and creating the read-only AWS role Sentinel is designed to assume.

Read [README.md](#) first — you need the service running on AWS, because onboarding asks for the real account ID and region it runs in.

Before you start

- `timeout-service` is deployed and answering on its Elastic IP.
- Its CloudWatch log group exists and contains at least one `upstream.timeout` event.
- You can sign in to Sentinel. If you have no organization yet, signing up founds one and makes you its admin; if someone sent you an invitation link, that link joins you to theirs.
- This repository (or your fork of it) is on GitHub, because onboarding requires an `owner/name` repository.

What Sentinel does with this today

Be clear on this before you start, so nothing below looks like a failure:

- Sentinel **does not verify** the AWS or GitHub connection. Both will show as `unverified` after registration, and that is the current state of the product.
- There is **no investigation provider**. A check you start will reach a terminal `failed` state with the message *"No investigation provider is configured. AWS evidence collection and model diagnosis are not available yet, so this run was not investigated."* That is the worker being honest, not a broken deployment.
- Sentinel has **no field for a log group name**. The log group you created is recorded here and tagged in AWS so a later phase can find it; nothing reads it now.

What onboarding *does* give you is a real application record, a real AWS connection row with a server-generated external ID, and a run whose every state transition is stored in the database and streamed to the run page.

Step 1 — Collect the values

From your deploy shell:

```
aws sts get-caller-identity --query Account --output text # the 12-digit account ID
echo "$AWS_REGION"
echo "http://$PUBLIC_IP"
```

Sentinel field	Value for this app	Rule
Application name	Timeout Service	1–120 characters
Environment	production	production , staging or development
Repository	<your-org>/sentinel-core	must look like owner/name
Default branch	main	1–255 characters
Repository access	read-only	see the note below
Account alias	sandbox	whatever you call this AWS account
Account ID	the 12 digits above	exactly 12 digits
Region	us-east-1	must match where you launched the instance
Owning team	Platform	1–120 characters
Notify channel	#timeout-service-oncall	must start with #

Pick `read-only` for repository access. `read-and-pr` describes a pull-request flow that is not built yet, and there is nothing in this sample worth proposing a fix to.

Step 2 — Register the application

1. Sign in and go to **Fleet → Onboard an application** (`/fleet/new`).
2. Work through the three steps — repository, AWS access, ownership — using the table above.
3. The AWS step shows a role ARN it will use rather than asking for one. It is always `arn:aws:iam::<account-id>:role/SentinelReadOnly` ; the role name is fixed.
4. Submit. The success screen shows two values you need next:
 - the **role ARN**
 - the **external ID** — a UUID generated by the server, never chosen by you, because its whole job is to keep someone else's Sentinel tenant from using your trust policy.

Copy the external ID now. Export it in your deploy shell:

```
export SENTINEL_EXTERNAL_ID='<paste the external ID>'
```

Step 3 — Create the SentinelReadOnly role

Sentinel will assume this role from the AWS principal its workers run as. Find that principal in the account hosting your Sentinel deployment — it is the instance role attached to the Sentinel EC2 instance:

```
# Run against the Sentinel deployment's account, not the sample app's.
aws sts get-caller-identity --query Arn --output text
```

Then, back in the account that hosts `timeout-service` :

```
export SENTINEL_PRINCIPAL_ARN='arn:aws:iam::<sentinel-account-id>:role/<sentinel-instance-role>'

cat > /tmp/sentinel-trust.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "${SENTINEL_PRINCIPAL_ARN}" },
    "Action": "sts:AssumeRole",
    "Condition": {
      "StringEquals": { "sts:ExternalId": "${SENTINEL_EXTERNAL_ID}" }
    }
  }]
}
JSON

aws iam create-role \
  --role-name SentinelReadOnly \
  --description 'Read-only access for Sentinel Ops investigations' \
  --max-session-duration 3600 \
  --assume-role-policy-document file:///tmp/sentinel-trust.json
```

Both halves of the condition matter. The principal says *who* may assume the role; the external ID says *on whose behalf*. Without the external ID condition, anyone who can persuade Sentinel to call `AssumeRole` with your account number could read your logs — the confused deputy problem the external ID exists to close.

Now the permissions. Read-only, and scoped to the one log group wherever the API allows it:

```

cat > /tmp/sentinel-readonly.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadApplicationLogs",
      "Effect": "Allow",
      "Action": [
        "logs:FilterLogEvents",
        "logs:GetLogEvents",
        "logs:DescribeLogStreams"
      ],
      "Resource": "arn:aws:logs:${AWS_REGION}:${ACCOUNT_ID}:log-group:${LOG_GROUP}:*"
    },
    {
      "Sid": "DiscoverAndQueryLogs",
      "Effect": "Allow",
      "Action": [
        "logs:DescribeLogGroups",
        "logs:StartQuery",
        "logs:StopQuery",
        "logs:GetQueryResults"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadMetricsAndAlarms",
      "Effect": "Allow",
      "Action": [
        "cloudwatch:GetMetricData",
        "cloudwatch:GetMetricStatistics",
        "cloudwatch:ListMetrics",
        "cloudwatch:DescribeAlarms",
        "cloudwatch:DescribeAlarmHistory"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DescribeCompute",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeInstanceStatus",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeAddresses",
        "tag:GetResources"
      ],
      "Resource": "*"
    }
  ]
}
JSON

aws iam put-role-policy \
  --role-name SentinelReadOnly \
  --policy-name SentinelReadOnlyAccess \
  --policy-document file:///tmp/sentinel-readonly.json

```

DescribeLogGroups , StartQuery and the CloudWatch metric actions do not support resource-level permissions, so they are granted on *. Everything that can be scoped is scoped, and nothing in this policy can change state. There is no logs:PutLogEvents , no ec2:*Modify* , no iam:*

Confirm the trust policy actually works:

```

aws sts assume-role \
  --role-arn "arn:aws:iam:${ACCOUNT_ID}:role/SentinelReadOnly" \
  --role-session-name connectivity-check \
  --external-id "$SENTINEL_EXTERNAL_ID" \
  --query 'AssumedRoleUser.Arn' --output text

```

Run that from the Sentinel principal, not from your admin identity. `AccessDenied` from your own laptop credentials is the expected and correct answer — it means the principal condition is doing its job.

Step 4 — Start a check

1. Go to **Ops agent** → **New check** (`/opsagent/new`).
2. Pick **Timeout Service** and set severity to `high` .
3. Describe the symptom in the text box. The first line becomes the incident title, so lead with the headline:

```
Checkout orders returning 504
GET /api/orders/:orderId started returning 504 gateway_timeout. The ledger call is
exceeding the 3000ms request deadline. CloudWatch shows upstream.timeout events in
/sentinel-sample/timeout-service.
```

4. Leave the depth on **Full investigation**, or pick **Quick look** — neither reads anything yet, but the choice is recorded on the run.
5. Submit. The run page opens and streams every state the database has recorded.

The run will end `failed` with the “No investigation provider is configured” message. What you should see working is everything around it: the run exists, its events arrive in order, the report is versioned, and the page reflects exactly what is stored.

If you want the application to actually be broken while you do this, flip it first:

```
ssh -i ~/.ssh/timeout-service.pem ec2-user@"$PUBLIC_IP" \
"curl -s -X POST localhost/admin/chaos \
  -H \"authorization: Bearer \$(cat /opt/timeout-service/chaos-token)\" \
  -H 'content-type: application/json' \
  -d '{\"latencyMs\":5000}'"
```

Reset it with `{\"latencyMs\":200}` when you are done, or the CloudWatch alarm will keep firing.

Record these for later

Sentinel has nowhere to store most of this yet. Keep it somewhere you will find it again when the investigation provider ships.

Fact	Where it lives
Log group	<code>/sentinel-sample/timeout-service</code>
Metric namespace	<code>SentinelSample/TimeoutService</code>
Timeout metric	<code>UpstreamTimeouts</code> , from the <code>{ \$.event = "upstream.timeout" }</code> filter
Alarm	<code>timeout-service-upstream-timeouts</code>
Role Sentinel assumes	<code>arn:aws:iam::<account-id>:role/SentinelReadOnly</code>
External ID	shown once on the onboarding success screen; also on the application's detail page
Failing route	<code>GET /api/orders/:orderId</code>
Deadline	<code>REQUEST_DEADLINE_MS</code> , 3000ms by default

Tear down the role

```
aws iam delete-role-policy --role-name SentinelReadOnly --policy-name SentinelReadOnlyAccess
aws iam delete-role --role-name SentinelReadOnly
```

The rest of the AWS resources are removed by the teardown section in [README.md](#).